

Report on DeepSDF: Learning Continuous Signed Distance Functions for Shape Representation

Adrian Steffan
Technical University of Munich
adrian.steffan@tum.de

Abstract

Existing 3D shape representations for deep learning trade off between fidelity, efficiency, and the ability to model arbitrary topology. DeepSDF aims to eliminate this trade-off by training a deep network to represent a class of shapes using conditioned signed distance functions. They train both the model and the conditioning latent codes jointly using an auto-decoder architecture that allows inference to run on various input modalities. DeepSDF’s ability to represent previously unseen shapes and complete shapes from partial observations exceeds the existing baselines for these tasks qualitatively and quantitatively. Though we discuss limitations of their work, DeepSDF’s novel approach addresses the limitations of prior 3D representations and pioneers a field-wide adoption of implicit 3D shape representations.

1. Introduction

Machine learning systems that can reason about 3D geometry increasingly drive applications spanning robotics, autonomous driving, and generative content creation. Yet 3D shape representation remains a foundational problem in deep learning: the choice of representation determines what shapes can be modeled, what tasks are viable, and how compactly the model scales. The task of representation can be phrased as follows: given 3D geometry observations (e.g. dense surface samples, partial depth scans, or point clouds), produce a learned representation that faithfully captures the shape’s geometry. Existing approaches have to make various trade-offs: voxel-based methods scale cubically with resolution and lose fine detail, mesh-based methods are constrained to restricted topologies, and point-cloud methods cannot represent closed surfaces. The authors of DeepSDF aim to provide a representational method that addresses these limitations while still fully supporting downstream reconstruction, completion, and generative use. They thus represent 3D shapes with a learned, continuous implicit field, i.e. a signed distance function. DeepSDF parameterizes the SDF

with a deep network conditioned on a per-shape latent code, so that one model can represent an entire class of shapes. These codes are learned jointly with the network weights during training in an auto-decoder architecture, which allows the model to flexibly handle various input modalities during inference. The paper’s approach, along with Occupancy Networks and IM-NET, was among the first to use implicit representations in 3D deep learning, laying the foundation for this new branch of research. Our report is structured as follows: ‘Sec. 2’ briefly highlights prior work; ‘Sec. 3’ presents the implicit representation and auto-decoder formulation; ‘Sec. 4’ reports the headline results on shape representation and completion; ‘Sec. 5’ gives our critical analysis.

2. Related Work

Existing, explicit 3D representations: Point-cloud methods like PointNet [13] treat shapes as unordered sets of 3D points, which cannot represent closed surfaces. Voxel-based approaches like OGN [16] and 3D-EPN [6] discretize shapes onto a regular grid, scaling cubically with resolution and losing fidelity. Mesh-based methods like AtlasNet [8] parameterize the surface using fixed templates or sphere mappings, which constrains the topologies they can represent. DeepSDF represents shapes continuously as a learned implicit field, avoiding these trade-offs.

Similar ideas at CVPR 2019: Occupancy Networks [10] train a binary classifier that predicts the probability of a query point being inside the shape, and IM-NET [3] uses a similar inside/outside formulation for generative modeling. Both use implicit representations, but they rely on traditional encoder-decoder architectures, whereas DeepSDF drops the encoder entirely with its auto-decoder formulation.

Follow-ups extending DeepSDF: DeepLS [1] and Local Implicit Grids [9] replace the single global code with per-region implicit fields, allowing the model to scale to whole scenes. NeRF [11] carried the implicit-field idea over to view synthesis with differentiable volumetric rendering, and MonoSDF [17] applied SDF-based reconstruction to monocular inputs. Even modern generative pipelines like

3DShape2VecSet [18] use implicit-field representations as the backbone for diffusion-based 3D generation. The breadth of these follow-ups shows how DeepSDF laid the groundwork for a line of future research. Implicit fields have not fully displaced explicit representations, however: methods like MeshGPT [15] have shown that direct mesh generation via autoregressive transformers can be competitive in reconstruction fidelity while producing more efficient meshes.

3. Method

DeepSDF represents a class of 3D shapes implicitly by learning an SDF conditioned on shape-specific latent codes learned alongside the network weights. To represent new shapes or complete partial ones at test time, DeepSDF takes point samples from the input shape and searches for the latent code whose SDF best matches the samples, after which the SDF for that code can be used to reconstruct the object.

SDFs are defined as continuous scalar fields $f : \mathbb{R}^3 \rightarrow \mathbb{R}$ where $|f(\mathbf{x})|$ is the distance from $\mathbf{x}$ to the nearest surface point. Negative values indicate points inside the shape, positive values indicate points outside the shape, and the zero level-set defines the surface. Such an SDF can be represented by a neural network f_θ that predicts distance values from a query point $\mathbf{x}$. Formula (1) shows a possible loss function for such a network, with clamping function $\text{clamp}(x, \delta) := \min(\delta, \max(-\delta, x))$ to focus the training budget on the points near the surface.

$$\mathcal{L}(\hat{s}, s) = |\text{clamp}(\hat{s}, \delta) - \text{clamp}(s, \delta)| \quad (1)$$

However, training one network per shape is hardly useful. Thus, DeepSDF adds a shape-specific latent code $\mathbf{z}$ as an additional input which conditions the network to act as the SDF for the given shape. This architecture is shown in Fig. 1 and enables DeepSDF to learn an entire family of shapes by training a single network together with one latent code per training shape.

The shape codes are organized into a latent space during training time using an auto-decoder architecture. Traditional auto-encoders use both an encoder and decoder that are jointly trained to compress an input into a latent representation and then reconstruct it. Auto-decoders drop the encoder component, and instead assign every training shape i an (initially random) latent code $\mathbf{z}_i$ that is then directly optimized alongside the decoder’s weights using gradient descent. This approach has two benefits: 1) For DeepSDF’s inference tasks, the encoder would be unused at test time, making its training wasted compute. 2) Traditional auto-encoders have strict input format requirements, or need to use workarounds to fix them like in the case of PointNet [13], whereas auto-decoders are modality-agnostic during inference.

The decoder weights and the latent codes are optimized using the following objective function:

Method	chair	plane	table	lamp	sofa
AtlasNet-Sph. [8]	0.752	0.188	0.725	2.381	0.445
AtlasNet-25 [8]	0.368	0.216	0.328	1.182	0.411
DeepSDF	0.204	0.143	0.553	0.832	0.132

Table 1. Chamfer distance (mean, $\times 10^3$) for shape representation on unseen ShapeNet test shapes; lower is better. Numbers reproduced from [12], Tab. 3.

$$\arg \min_{\theta, \{\mathbf{z}_i\}} \sum_{i=1}^N \left(\sum_{j=1}^K \mathcal{L}(f_\theta(\mathbf{z}_i, \mathbf{x}_j), s_j) + \frac{1}{\sigma^2} \|\mathbf{z}_i\|_2^2 \right) \quad (2)$$

For each training shape, K point samples $(\mathbf{x}_j, s_j)$ with ground-truth SDF values are drawn, and the loss sums over the deviations from all samples of all shapes. Additionally, the objective function introduces an $L2$ regularization term for the latent codes $\mathbf{z}$, which corresponds to placing a zero-mean isotropic Gaussian prior on the codes. This prior concentrates the codes near the origin and prevents them from drifting during training, in turn making space traversal at inference time more robust.

During inference, DeepSDF can represent known and unknown shapes. The input can be of any modality that allows for the sampling of points near the surface. Once the set of points is extracted, the task of the inference pipeline is to find the code whose conditioned SDF minimizes the deviation from these points. Specifically, gradient descent is run on the same objective as the main training loop, using the samples as training input and optimizing the latent codes while keeping the network weights frozen. As this approach does not dictate a specific number or location of points, it can be used to find latent codes from partial observations. The procedure in this case simply uses the surface points received from the visible portion of the object to find the code whose SDF best matches the visible samples, with the Gaussian prior keeping the optimizer within the region of plausible shapes seen during training. The same compact latent space also enables smooth interpolation between shapes: a point between two codes yields an SDF describing a plausible intermediate shape. Once a code $\hat{\mathbf{z}}$ for a shape has been recovered, an explicit representation can be extracted from the conditioned SDF by running marching cubes (for a mesh) or by ray tracing.

4. Experiments & Results

The experiments use five shape categories of ShapeNet [2] as data (chair, plane, table, lamp, sofa). For each shape, 250K points are sampled on the surface and perturbed twice with Gaussian noise to produce 500K near-surface samples, alongside 25K uniformly-distributed samples inside the unit

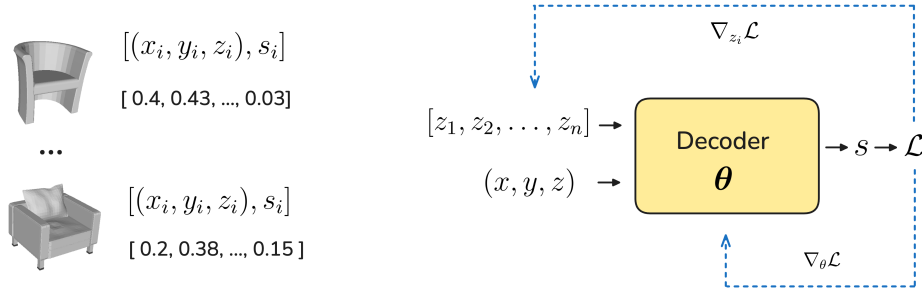


Figure 1. The DeepSDF auto-decoder architecture. Each training shape (left) is paired with its own latent code (shown as example values). A single shared decoder f_θ takes a latent code $[z_1, \dots, z_n]$ and a query point (x, y, z) as input, and predicts the signed distance value s . Both the latent codes $\{z_i\}$ and the decoder weights θ are optimized jointly via gradient descent on the reconstruction loss $\mathcal{L}$.

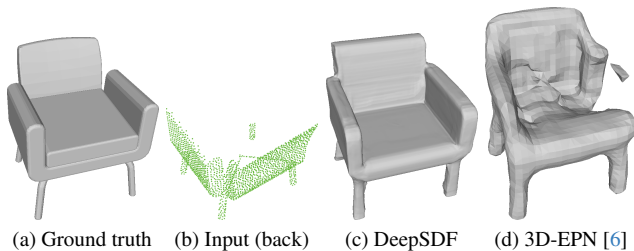


Figure 2. Shape completion from a partial depth scan ((b)). DeepSDF ((c), novel view) matches the ground truth ((a)); 3D-EPN ((d)) shows blocky voxel artifacts. Adapted from [12], Fig. 8.

sphere. SDF values are computed at each point. The decoder is an 8-layer MLP with 512-dim hidden layers, ReLU, a skip connection to layer 4, weight normalization, and tanh output. The latent codes are either 256 dimensional for shape representation or 128 dimensional for completion experiments. Training uses Adam; the objective function uses $\delta = 0.1$ (clamp) and $\sigma^2 = 10^{-2}$ (prior); latent codes are initialized from $\mathcal{N}(0, 0.01^2)$. The main measures used for the experiments are Chamfer Distance (CD) [7], and Earth Mover’s Distance (EMD) [14]. The points used to calculate these measures are sampled probabilistically from the compared objects.

To test whether the auto-decoder can accurately represent shapes outside the training set, DeepSDF is evaluated on unseen items from ShapeNet. For each test shape, the full set of SDF samples is supplied as input and a latent code $\hat{z}$ is recovered via gradient descent. The resulting conditioned SDF is then turned into a mesh via marching cubes. Tab. 1 reports Chamfer distance per category against AtlasNet [8]. AtlasNet wins on *tables* where simpler topology makes its parametric-mesh constraint less limiting. In the other, topologically more complex shapes, DeepSDF outperforms it. Qualitatively, DeepSDF also resolves topological details that AtlasNet’s parametric mesh tends to leave as continuous surfaces (Fig. 3). The results validate that the optimization-based encoding produces plausible representations even for

shapes outside the training set. Shapes that are uncommon or have high-frequency geometry lead to worse reconstructions, as shown in Fig. 4.

Next, the paper tests how well DeepSDF completes shapes from partial observations. They gather surface points from one side of an object via a virtual camera and then run the same inference steps as for shape representation to retrieve the best fitting latent code. Fig. 2 shows a qualitative comparison between reconstructions of DeepSDF and 3D-EPN. DeepSDF comes a lot closer to the original shape, which the authors attribute to the strong shape priors encoded in the network. The Gaussian prior keeps the optimizer within plausible-shape territory, whereas 3D-EPN, doing direct voxel-to-voxel prediction, has no comparable latent regularization. DeepSDF also beats 3D-EPN quantitatively across all metrics and categories, e.g. the CD median for the chair category is 1.28 for DeepSDF vs. 2.25 for 3D-EPN. Even though DeepSDF had no completion-specific training, it outperforms the baseline in these measures, demonstrating the flexibility of their auto-decoder architecture.

To further confirm how robust the completion procedure is, the authors add noise to the partial input observations, simulating consumer-grade depth-camera noise. They take their scans from the ShapeNet planes test set and perturb the points with Gaussian noise of increasing standard deviation (0.01, 0.02, 0.03, 0.05). While the CD between the perturbed and ground truth depth observations grows superlinearly ($\approx 6.5 \times 10^3$ at $\alpha = 0.05$), the completion’s CD against the ground-truth mesh grows only linearly (≈ 1.3 at $\alpha = 0.05$). The learned latent space and the Gaussian prior put on the codes in the objective function pull the optimizer towards plausible shapes instead of overfitting to the noise.

Finally, the smooth structure of the latent space allows DeepSDF to generate new shapes from it by interpolating between the codes of known objects. Qualitative results from both the paper and our own training runs are shown in Fig. 5.

5. Critical Analysis

5.1. Strengths

DeepSDF’s auto-decoder formulation generalizes across input modalities in a single architecture: it can perform inference on dense surface samples, partial depth scans, and noisy point clouds (e.g. from meshes, depth cameras, or LiDAR) without retraining, whereas methods like 3D-EPN require completion-specific encoders. The paper also unifies shape representation, completion, and latent-space interpolation under one model, thus framing these as different inferences on the same representation rather than as separate problems. Furthermore, the L2 regularizer is grounded in a probabilistic derivation rather than empirical tuning. The Gaussian prior framing on latent codes makes it easier to reason about the behavior of the model during inference. Finally, the implicit representation’s memory footprint only scales with the dimension of the latent space rather than spatial resolution, allowing thousands of shapes to be stored in a few MB.

5.2. Weaknesses and Limitations

Two major limitations are already mentioned by the authors: Inference requires gradient descent rather than a simple forward pass like AtlasNet, taking roughly 9.72 seconds per shape and ruling out real-time applications. Furthermore, the method assumes shapes in a canonical pose, so handling rotated inputs requires additionally optimizing for an SE(3) transformation, further increasing the inference time.

DeepSDF makes a limiting structural choice that the paper does not mention: The single global latent code per shape assigns uniform network capacity to every region, regardless of which regions actually require that level of detail. Additionally, a handful of methodological concerns weaken the paper’s empirical claims: The ablation studies are constructed somewhat arbitrarily (500 chairs, 100 cars, MNIST), giving the authors substantial researcher degrees of freedom. Reproducibility is also hindered by an incomplete public code repository (e.g. no code for shape completion) whose hyperparameter values do not match the paper’s reported numbers. Finally, the paper uses probabilistic evaluation metrics but reports them only as point estimates: CD and EMD both depend on randomly sampled point sets (with EMD using only 500 samples), and the paper reports means and medians without variances or confidence intervals. This omission makes it difficult to separate actual model differences from sampling noise.

5.3. Comparison with Related Work

3DShape2VecSet [18] mirrors DeepSDF’s implicit representation idea but uses a set of multiple latent vectors per shape instead of a single global one. This extension directly addresses DeepSDF’s limitation of the uniform detail budget: instead of encoding the entire shape in one global code, mul-

iple latent vectors can carry different aspects of the shape’s information. The method uses the same continuous implicit fields conditioned on learned codes, but the set-of-latents approach achieves markedly better shape reconstruction quality on ShapeNet benchmarks than single-global-latent methods.

MeshGPT [15] demonstrates an alternative paradigm: Rather than learning an implicit field that must then be meshed via marching cubes, it autoregressively generates triangle meshes using a GPT-style decoder-only transformer. The output is more compact than DeepSDF-style marching-cubes output, with sharper edges and triangulation patterns resembling human-crafted meshes rather than the dense geometry typical of marching cubes. The contrast suggests DeepSDF’s implicit-field paradigm is one option among several: it remains a strong choice when continuous shape interpolation and topology flexibility matter, but direct mesh generation has proven competitive when output compactness and downstream mesh editability are important.

5.4. What I Would Improve

I would fix the mentioned methodological shortcoming by properly computing variances for the evaluation metrics. CD and EMD produce variance through their random point sampling; running the evaluation multiple times with fresh samples and reporting standard deviations or confidence intervals would let readers judge whether reported gaps reflect genuine model differences or are just sampling noise. Several of the shape completion margins on the chair category are small enough for this to matter: DeepSDF’s wins over 3D-EPN on EMD (0.071 vs 0.084) or mesh accuracy (0.049 vs 0.059) are gaps that could potentially be attributable to sampling noise. Additionally, bootstrapping over test set and rerunning the metric comparisons would yield confidence intervals for model differences and enable hypothesis tests on which of DeepSDF’s metric improvements over the baselines is statistically significant.

Lastly, I would evaluate DeepSDF and baseline on more topologically complex shapes, as the paper highlights the ability to represent these as a central strength of their method. The chosen ShapeNet categories (chairs, planes, tables, lamps, sofas) are mostly topologically tame or have low-frequency surfaces, which undersells DeepSDF’s claimed advantage over the baseline methods. Running the experiments on shapes with multiple holes or higher-frequency surfaces would either show an even wider gap between the paper and the baseline, or reveal hidden failure modes.

5.5. Reproduction Notes

I trained DeepSDF on 209 chairs from the Amazon Berkeley Objects dataset [4] for 1000 epochs on an Nvidia RTX 5090. Latent-space interpolation between codes produced smooth chair morphs consistent with the paper’s interpolation results. See Appendix Secs. A to C for further details.

References

- [1] Rohan Chabra, Jan E. Lenssen, Eddy Ilg, Tanner Schmidt, Julian Straub, Steven Lovegrove, and Richard Newcombe. Deep local shapes: Learning local sdf priors for detailed 3d reconstruction. In *ECCV*, page 608–625, Berlin, Heidelberg, 2020. Springer-Verlag. 1
- [2] Angel X. Chang, Thomas Funkhouser, Leonidas Guibas, Pat Hanrahan, Qixing Huang, Zimo Li, Silvio Savarese, Manolis Savva, Shuran Song, Hao Su, Jianxiong Xiao, Li Yi, and Fisher Yu. Shapenet: An information-rich 3d model repository, 2015. 2
- [3] Zhiqin Chen and Hao Zhang. Learning implicit fields for generative shape modeling. In *CVPR*, pages 5939–5948, 2019. 1
- [4] Jasmine Collins, Shubham Goel, Kenan Deng, Achleshwar Luthra, Leon Xu, Erhan Gundogdu, Xi Zhang, Tomas F. Yago Vicente, Thomas Dideriksen, Himanshu Arora, Matthieu Guillaumin, and Jitendra Malik. ABO: Dataset and benchmarks for real-world 3D object understanding. In *CVPR*, pages 21126–21136, 2022. 4, 5, 7
- [5] Mauro Comi. Deepsdf-minimal. <https://github.com/maurock/DeepsDF/>, 2023. 5
- [6] Angela Dai, Charles Ruizhongtai Qi, and Matthias Nießner. Shape completion using 3d-encoder-predictor cnns and shape synthesis. In *CVPR*, 2017. 1, 3
- [7] Haoqiang Fan, Hao Su, and Leonidas J. Guibas. A point set generation network for 3D object reconstruction from a single image. In *CVPR*, pages 2463–2471, 2017. 3
- [8] Thibault Groueix, Matthew Fisher, Vladimir G. Kim, Bryan Russell, and Mathieu Aubry. AtlasNet: A Papier-Mâché Approach to Learning 3D Surface Generation. In *CVPR*, 2018. 1, 2, 3
- [9] Chiyu Jiang, Avneesh Sud, Ameesh Makadia, Jingwei Huang, Matthias Nießner, and Thomas Funkhouser. Local implicit grid representations for 3d scenes. In *CVPR*, pages 6000–6009, 2020. 1
- [10] Lars Mescheder, Michael Oechsle, Michael Niemeyer, Sebastian Nowozin, and Andreas Geiger. Occupancy networks: Learning 3d reconstruction in function space. In *CVPR*, 2019. 1
- [11] Ben Mildenhall, Pratul P. Srinivasan, Matthew Tancik, Jonathan T. Barron, Ravi Ramamoorthi, and Ren Ng. Nerf: representing scenes as neural radiance fields for view synthesis. *Commun. ACM*, 65(1):99–106, 2021. 1
- [12] Jeong Joon Park, Peter Florence, Julian Straub, Richard Newcombe, and Steven Lovegrove. Deepsdf: Learning continuous signed distance functions for shape representation. In *CVPR*, 2019. 2, 3, 6, 7
- [13] Charles R. Qi, Hao Su, Kaichun Mo, and Leonidas J. Guibas. PointNet: Deep learning on point sets for 3D classification and segmentation. In *CVPR*, pages 652–660, 2017. 1, 2
- [14] Yossi Rubner, Carlo Tomasi, and Leonidas J. Guibas. A metric for distributions with applications to image databases. In *ICCV*, pages 59–66, 1998. 3
- [15] Yawar Siddiqui, Antonio Alliegro, Alexey Artemov, Tatiana Tommasi, Daniele Sirigatti, Vladislav Rosov, Angela Dai, and Matthias Nießner. MeshGPT: Generating triangle meshes with decoder-only transformers. In *CVPR*, pages 19615–19625, 2024. 2, 4
- [16] Maxim Tatarchenko, Alexey Dosovitskiy, and Thomas Brox. Octree generating networks: Efficient convolutional architectures for high-resolution 3d outputs. In *ICCV*, 2017. 1
- [17] Zehao Yu, Songyou Peng, Michael Niemeyer, Torsten Sattler, and Andreas Geiger. MonoSDF: Exploring monocular geometric cues for neural implicit surface reconstruction. In *NeurIPS*, 2022. 1
- [18] Biao Zhang, Jiapeng Tang, Matthias Nießner, and Peter Wonka. 3dshape2vecset: A 3d shape representation for neural fields and generative diffusion models. *ACM TOG*, 42(4), 2023. 2, 4

A. Reproduction Setup

Initially, I planned to rerun the paper’s pipeline to compute proper variances and confidence intervals for the reported metrics, as proposed in Sec. 5.4. However, the official code repository proved incomplete (no shape-completion code; hyperparameters that did not match the paper’s reported numbers), and parts of the pipeline relied on C++ libraries that were hard to reproduce on my end. In preliminary testing, the metric numbers from my own (deviating) implementation differed from the paper’s (even when running ShapeNet), so a reanalysis would not have been comparable. I therefore focused on qualitative demonstrations using my own retraining of DeepSDF, built on an open-source PyTorch re-implementation [5]. The code used to create the examples in this appendix is available at¹.

I ran all code on a single NVIDIA RTX 5090 (32 GB); dependency versions can be found in the repository. The training data consists of 209 chair-class shapes drawn from Amazon Berkeley Objects [4], sampled with a fixed random seed from ABO’s CHAIR, STOOL_SEATING, and BENCH product types. Each mesh is normalised to a unit bounding box, and sampled at 23,000 SDF query points per shape. The decoder architecture follows the paper exactly.

Departures from the paper. The adapted reimplementations differs in places where I had to make tradeoffs for my small compute scale or where the released code contradicted the paper: (i) 209 ABO chairs instead of 3,281 ShapeNet-V2 chairs; (ii) 23,000 SDF samples per shape via uniform/surface sampling, instead of the paper’s $\sim 500,000$ camera-cast samples; (iii) a linear latent regulariser $\sigma^2 \|z\|_2$ with $\sigma = 0.01$, inherited from the existing reimplementations [5] rather than the paper’s quadratic form $(1/\sigma^2) \|z\|_2^2$.²

Demo inferences freeze the decoder and optimise z from the mean of the trained latents with Adam, then run marching cubes at 256^3 . Full per-demo configurations and the released model checkpoint can be found in the repository.

¹<https://github.com/adriansteffan/deepsdf-demo>

²The inherited linear form has the same sign as the paper’s quadratic prior (pulling toward $z = 0$) but is considerably weaker.

B. Additional Results

Fig. 5 shows my reproduction of the paper’s interpolation experiment. Similarly to the paper, the interpolation is smooth and passes plausible intermediate states between the two chairs from the training data. Even though my training setup and dataset diverged from the main paper, it was still able to reproduce similar results, indicating that the underlying architecture is robust against small implementation deviations and the qualitative results are reproducible with similar datasets.

Fig. 6 highlights the task flexibility of DeepSDF. In this demo, I used DeepSDF to perform prior-conditioned pseudo-upscaling by sampling surface points from a low-resolution version of one of the chairs in the dataset and then using the gradient descent optimization to find the best fitting shape code in the latent space. The resulting render is the closest “high-resolution” version of the input shape that can be represented by the model. As low-resolution upscaling can be seen as latent-code inference from a sparsely-sampled surface, the same operation DeepSDF performs for representation and completion, this use case emerges naturally from the already existing inference pipeline with no additional training, a strength highlighted in the analysis.

C. Failure Cases

Marching-cubes surface artifacts. The reproductions in Fig. 5 show visible ripples on the flat surfaces of every chair (most prominent on seat backs and armrests). These are assumed to be discretization artifacts from marching cubes. They are not specific to this training run but likely a fundamental side-effect of converting a continuous SDF into an explicit mesh by sampling at a finite grid resolution. The paper sidesteps the problem in its teaser by raycasting the SDF directly rather than extracting a mesh, an option the authors discussed in their methodology but didn’t include in the official code repository.

Thin-feature collapse. The rightmost chair in our reproduction row shows jagged artefacts in the back where we would expect clean parallel rails. DeepSDF struggles with such thin parallel surfaces, resulting in low-frequency noise. This can be attributed to the fact that the network’s bandwidth is finite, which is why it struggles to accurately represent narrow features that require the SDF to vary rapidly within a small subset of the 3D space. These visual inconsistencies compound with the marching-cubes step described above. The paper acknowledges similar high-frequency failure modes as shown in Fig. 4.

Rare Pokémon. Finally, in a discarded experiment I trained a DeepSDF model on 424 meshes extracted from 3D Pokémon games, one mesh per Pokémon. Fig. 7 shows an interpolation that was meant to represent an evolution like you would find in a Pokémon game, interpolating be-

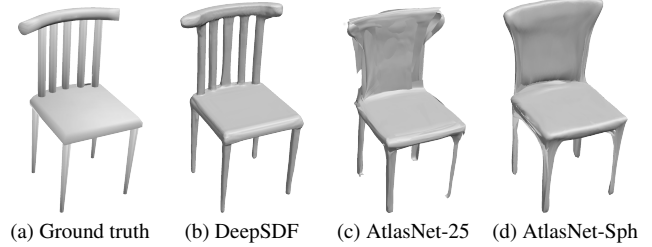


Figure 3. Qualitative reconstruction comparison on a test chair from ShapeNet. AtlasNet struggles with holes in the back of the chair, while DeepSDF’s continuous SDF representation easily handles arbitrary topology without modification. Adapted from [12], Fig. 6.



Figure 4. Failure modes of DeepSDF on uncommon or high-frequency test shapes. These examples likely fall in undertrained regions of the latent space. Adapted from [12], Fig. 7.

tween three evolution stages of a Pokémon family: Charmander, Charmeleon, and Charizard³. While Charmander and Charmeleon are vaguely recognisable, the reproduction of Charizard fails, specifically for the wings and the arms that are thinner than those of its predecessors. I attribute this to the fact that the training data had few Pokémon with a wing structure like Charizard, meaning the model struggled to learn these details, while builds like Charmander (small character standing on two legs) were much more common in the dataset, and thus got balanced representation in the latent space. While we should be careful to draw conclusions from this limited experiment, it shows that DeepSDF struggles to faithfully represent rare/unique elements, consistent with the paper’s own failure cases on uncommon shapes (Fig. 4).

³[https://bulbapedia.bulbagarden.net/wiki/Charmander_\(Pok%C3%A9mon\)](https://bulbapedia.bulbagarden.net/wiki/Charmander_(Pok%C3%A9mon))

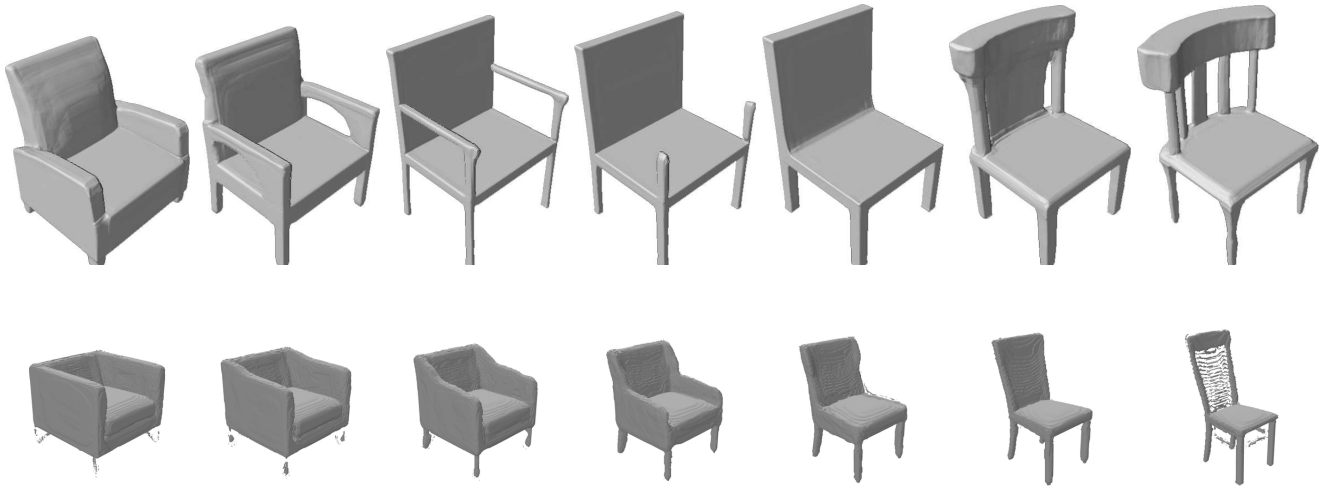


Figure 5. Top: latent-space interpolation between two chair shapes from [12], Fig. 1 (paper’s ShapeNet training). Bottom: our reproduction, trained on the Amazon Berkeley Objects chair subset [4], decoded via marching cubes at seven linearly-interpolated latent codes between the similar chair types.



Figure 6. Prior-conditioned pseudo upscaling. Starting from the Gaussian prior mean $z_0 = \mathbf{0}$, we optimize the latent code by gradient descent against surface samples from a low-resolution input chair (leftmost). Subsequent frames show the decoded mesh as the optimizer pulls z from the prior toward the target.



Figure 7. Latent space interpolation between the codes for the Pokémon Charmander, Charmeleon, and Charizard to visually mimic an “evolution”. Visible failure to represent Charizard’s wings as the shape shares little elements with the other members of the training data.